Дослідження transform (варіант 15)

Що робилось. Компілятор MSVC (Visual Studio), x64 Release, C++20, процесор має 16 логічних потоків. Дані: випадкові числа double з [1; 100], довжини 10^3, 10^5, 10^6, $5 \cdot 10^6$; збережені у файлах data_N.txt і однакові для всіх запусків. Дві операції: **швидка** (2x+1) і **повільна** (sin, cos, √, ln, exp). Режими: **/Od** (без оптимізації) та **/O2** (максимальна). Час — медіана з 5 запусків. Власний алгоритм ділить дані на K частин, кожну частину обробляє окремий потік звичайним transform, потім результати послідовно зводяться через сору. Правильність результату перевірена для кожного K.

1–2. Бібліотечний transform, N = $5 \cdot 10^6$, час у мілісекундах

Політика	**швидка, /Od**	**швидка, /O2**	**повільна, /Od**	**повільна, /O2**
без політики	11,5	4,6	170,0	149,3
seq	11,7	4,8	171,8	147,5
par	4,9	2,6	17,4	16,3
unseq	11,5	4,3	168,8	148,8
par_unseq	4,1	3,0	17,6	16,6

3. Власний алгоритм: найкраще K (потоків процесора — 16)

Операція, режим	**N = 10^3**	**N = 10^5**	**N = 10^6**	**N = $5 \cdot 10^6$**
повільна, /O2	2	10	16	**16**
повільна, /Od	1	4	32	25
швидка, /O2	1	1	4	**5**
швидка, /Od	1	5	3	21

Залежність часу власного алгоритму від K (режим /O2, мілісекунди)

K	**1**	**2**	**4**	**8**	**16**	**64**	**128**	**256**	**512**
повільна, N = $5 \cdot 10^6$	162,5	84,7	46,8	38,2	26,1	27,7	29,4	34,5	46,5
швидка, N = 10^3	0,19	0,25	0,38	0,55	0,69	2,39	4,85	9,52	19,47

Висновки

1. Оптимізація. /O2 прискорює швидку операцію у 2,5 раза (11,5→4,6 мс), а повільну майже ні (170→149 мс): там весь час іде на sin, cos, ln, exp.

2. Політики. seq і unseq працюють так само, як без політики. par і par_unseq прискорюють повільну операцію приблизно в 9–10 разів (≈16 мс замість ≈150–170 мс), швидку — лише у 2–3 рази, бо вона впирається в пам'ять. На малих даних (10^3 елементів) паралельні політики навпаки повільніші (8–15 мкс проти 0,2 мкс) через витрати на запуск потоків.

3. Найкраще K. Для повільної операції на великих даних найкраще K = 16, тобто **дорівнює кількості потоків процесора**. Для швидкої операції найкраще K значно менше (4–5), бо пам'ять не встигає за більшою кількістю потоків. Значення K=21, 25, 32 у режимі /Od — це випадкові коливання на майже рівній ділянці, а не справжній мінімум. Порівняно з K=1, оптимальне K дає прискорення у 6,2 раза для повільної операції і у 1,75 раза для швидкої.

4. Як росте час із K. Спочатку, поки K менше за кількість потоків, час зменшується приблизно як 1/K. Коли K стає більшим за кількість потоків, час зростає **лінійно**: кожен додатковий потік додає приблизно 40 мкс на його створення. Це видно в останньому рядку третьої таблиці: при K = 64, 128, 256, 512 час подвоюється (2,4 → 4,8 → 9,5 → 19,5 мс).

5. Власний алгоритм проти бібліотечного. Навіть із найкращим K він повільніший за par (26 проти 16 мс для повільної, 13 проти 2,6 мс для швидкої): потоки створюються заново при кожному виклику, а результат ще й копіюється з проміжних буферів.